

Subprograms - Concepts of Programming Languages

1. Fundamentals of Subprograms

Definition: A subprogram is a named collection of statements that can be invoked from elsewhere in a program.

Types:

- **Procedures:** Don't return values (e.g., `void` in C/Java)
- **Functions:** Return values

Example (Python):

```
# Function (returns value)
def add(a, b):
    return a + b

# Procedure (no return)
def print_message(msg):
    print(msg)

result = add(5, 3)      # result = 8
print_message("Hello") # prints: Hello
```

2. Parameters and Parameter Passing

2.1 Pass-by-Value

Concept: Copy of actual parameter is passed. Changes don't affect original.

Example (C):

```
void increment(int x) {
    x = x + 1; // Only local copy changed
}

int main() {
    int num = 5;
    increment(num);
    printf("%d", num); // Still prints 5
}
```

2.2 Pass-by-Reference

Concept: Address/reference of actual parameter is passed. Changes affect original.

Example (C++):

```

void increment(int &x) { // & means reference
    x = x + 1; // Original changed
}

int main() {
    int num = 5;
    increment(num);
    cout << num; // Prints 6
}

```

Example (Python - mutable objects):

```

def modify_list(lst):
    lst.append(4) # Original list modified

my_list = [1, 2, 3]
modify_list(my_list)
print(my_list) # [1, 2, 3, 4]

```

2.3 Pass-by-Result (Pass-by-Copy-Out)

Concept: No value passed in, but final value copied out after execution.

Conceptual Example (Ada-like):

```

procedure compute(out result: Integer) is
begin
    result := 42; -- Value assigned and copied out
end;

-- After call, caller's variable gets 42

```

2.4 Pass-by-Value-Result (In-Out)

Concept: Value copied in, modified, then copied back out.

Example (Ada):

```

procedure swap(in out x, y: Integer) is
    temp: Integer;
begin
    temp := x;
    x := y;
    y := temp;
end;

```

2.5 Pass-by-Name

Concept: Actual parameter is textually substituted (like macro). Evaluated each time used.

Conceptual Example (Algol-60 style):

```

procedure weird(name x, name i);
begin
    x := x + 1;

```

```
i := i + 1;
x := x + 1;
end;

A[1] := 0; i := 1;
weird(A[i], i);
// First x: A[1]++ -> A[1]=1
// Then i++      -> i=2
// Second x: A[2]++ -> A[2]=1 (different element!)
```

3. Local Referencing Environments

Concept: Variables visible within a subprogram (local, parameters, non-local).

Taught in a separate lecture

4. Overloaded Subprograms

Concept: Multiple subprograms with same name but different parameter types/counts.

Example (Java):

```
class Calculator {
    // Same name, different parameters
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

Calculator calc = new Calculator();
calc.add(5, 3);    // Calls int version: 8
calc.add(5.5, 3.2); // Calls double version: 8.7
calc.add(1, 2, 3); // Calls 3-parameter version: 6
```

5. Generic Subprograms

Concept: Subprograms that work with multiple types using type parameters (templates/generics).

Example (C++ Templates):

```
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

```

}

int x = max(5, 10);           // Works with int
double y = max(5.5, 3.2);    // Works with double
string z = max("abc", "xyz"); // Works with string

```

Example (Java Generics):

```

public class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}

Box<Integer> intBox = new Box<>();
intBox.set(123);

Box<String> strBox = new Box<>();
strBox.set("Hello");

```

Example (Python - naturally generic):

```

def find_max(items):
    max_item = items[0]
    for item in items:
        if item > max_item:
            max_item = item
    return max_item

find_max([1, 5, 3])           # Works with int
find_max([1.5, 3.2, 2.1])     # Works with float
find_max(["a", "z", "m"])     # Works with string

```

6. Closures

Concept: Function that captures variables from its enclosing scope.

Example (Python):

```

def make_multiplier(n):
    def multiply(x):
        return x * n # Captures 'n' from outer scope
    return multiply

times3 = make_multiplier(3)
times5 = make_multiplier(5)

print(times3(10)) # 30 (remembers n=3)
print(times5(10)) # 50 (remembers n=5)

```

7. Coroutines

Concept: Subprograms that can suspend execution and resume later, maintaining state.

Difference from Regular Functions:

- Functions: Run to completion, return once
- Coroutines: Can pause (yield) and resume multiple times

Example (Python - Generator as Simple Coroutine):

```
def simple_coroutine():
    print("Starting")
    yield 1
    print("Resuming")
    yield 2
    print("Ending")
    yield 3

gen = simple_coroutine()
print(next(gen)) # Starting, returns 1
print(next(gen)) # Resuming, returns 2
print(next(gen)) # Ending, returns 3
```

Example (Python - Producer/Consumer):

```
def producer():
    for i in range(5):
        print(f"Producing {i}")
        yield i

def consumer(gen):
    for value in gen:
        print(f"Consuming {value}")

consumer(producer())
# Output:
# Producing 0
# Consuming 0
# Producing 1
# Consuming 1
# ...
```

Practical Application (Infinite sequence generator):

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
```

```
fib = fibonacci()  
print(next(fib)) # 0  
print(next(fib)) # 1  
print(next(fib)) # 1  
print(next(fib)) # 2  
print(next(fib)) # 3
```